Step 1: Install Libraries and Import Dependencies

```
!pip install fairlearn

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from fairlearn.metrics import MetricFrame, selection_rate, true_positive_rate, false_positive_rate
from fairlearn.metrics import demographic_parity_difference, equalized_odds_difference, demographic_parity_ratio
```

Show hidden output

Step 2: Generate Synthetic Credit Data

```
np.random.seed(42)
n = 1000

# Protected Attribute: 0 = Unprivileged, 1 = Privileged
group = np.random.binomial(1, 0.5, n)
income = np.random.normal(50, 15, n) + (group * 10)
credit_score = np.random.normal(650, 50, n)

# Create binary target variable (Y)
y = (income + (credit_score / 10) + np.random.normal(0, 10, n) > 115).astype(int)
X = pd.DataFrame({'income': income, 'credit_score': credit_score})

# Train/Test Split (test_size corrected to 0.3)
X_train, X_test, y_train, y_test, A_train, A_test = train_test_split(
    X, y, group, test_size=0.3, random_state=42
)
```

Step 3: Train the Classifier

```
clf = RandomForestClassifier(random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
```

Step 4: Compute Group-Wise Performance Metrics

```
metrics = {
    'selection_rate': selection_rate,
    'TPR': true_positive_rate,
    'FPR': false_positive_rate
}

mf = MetricFrame(metrics=metrics, y_true=y_test, y_pred=y_pred, sensitive_features=A_test)
print('Group Metrics:\n', mf.by_group)
```

```
Group Metrics:
                     selection_rate       TPR       FPR
sensitive_feature_0
0                          0.516129  0.810127  0.210526
1                          0.779310  0.951456  0.357143
```

Step 5: Calculate Aggregate Fairness Metric Differences

```
dp_diff = demographic_parity_difference(y_test, y_pred, sensitive_features=A_test)
eq_odds_diff = equalized_odds_difference(y_test, y_pred, sensitive_features=A_test)
di_ratio = demographic_parity_ratio(y_test, y_pred, sensitive_features=A_test)

print(f'Demographic Parity Difference: {dp_diff:.4f}')
print(f'Equalized Odds Difference: {eq_odds_diff:.4f}')
print(f'Disparate Impact Ratio: {di_ratio:.4f}')
```

```
Demographic Parity Difference: 0.2632
Equalized Odds Difference: 0.1466
Disparate Impact Ratio: 0.6623
```

```
import matplotlib.pyplot as plt
import numpy as np

# Extract the group metrics from your existing MetricFrame (mf)
group_metrics = mf.by_group

# ========================================
# Visualization 1: Group-Wise Metrics
# ========================================
fig, ax = plt.subplots(figsize=(10, 6))
x = np.arange(len(group_metrics.columns))
width = 0.35

# Plot bars for Group 0 and Group 1
rects1 = ax.bar(x - width/2, group_metrics.loc[0], width, label='Unprivileged (Group 0)', color='#4C72B0')
rects2 = ax.bar(x + width/2, group_metrics.loc[1], width, label='Privileged (Group 1)', color='#C44E52')

# Labeling and Formatting
ax.set_ylabel('Rate Score')
ax.set_title('Group-Wise Fairness Metrics Comparison')
ax.set_xticks(x)
ax.set_xticklabels(['Selection Rate', 'True Positive Rate (TPR)', 'False Positive Rate (FPR)'])
ax.legend()
plt.ylim(0, 1.1)

# Annotate bars with exact values from your results
for rect in rects1 + rects2:
    height = rect.get_height()
    ax.annotate(f'{height:.4f}',
                xy=(rect.get_x() + rect.get_width() / 2, height),
                xytext=(0, 3),  # 3 points vertical offset
                textcoords="offset points",
                ha='center', va='bottom')

plt.tight_layout()
plt.show()

# ========================================
# Visualization 2: Aggregate Metrics vs. Legal/Ideal Thresholds
# ========================================
fig, ax = plt.subplots(figsize=(10, 4))
metrics_names = ['Demographic Parity Diff', 'Equalized Odds Diff', 'Disparate Impact Ratio']
# Using the variables you calculated in Step 5
metrics_values = [dp_diff, eq_odds_diff, di_ratio]

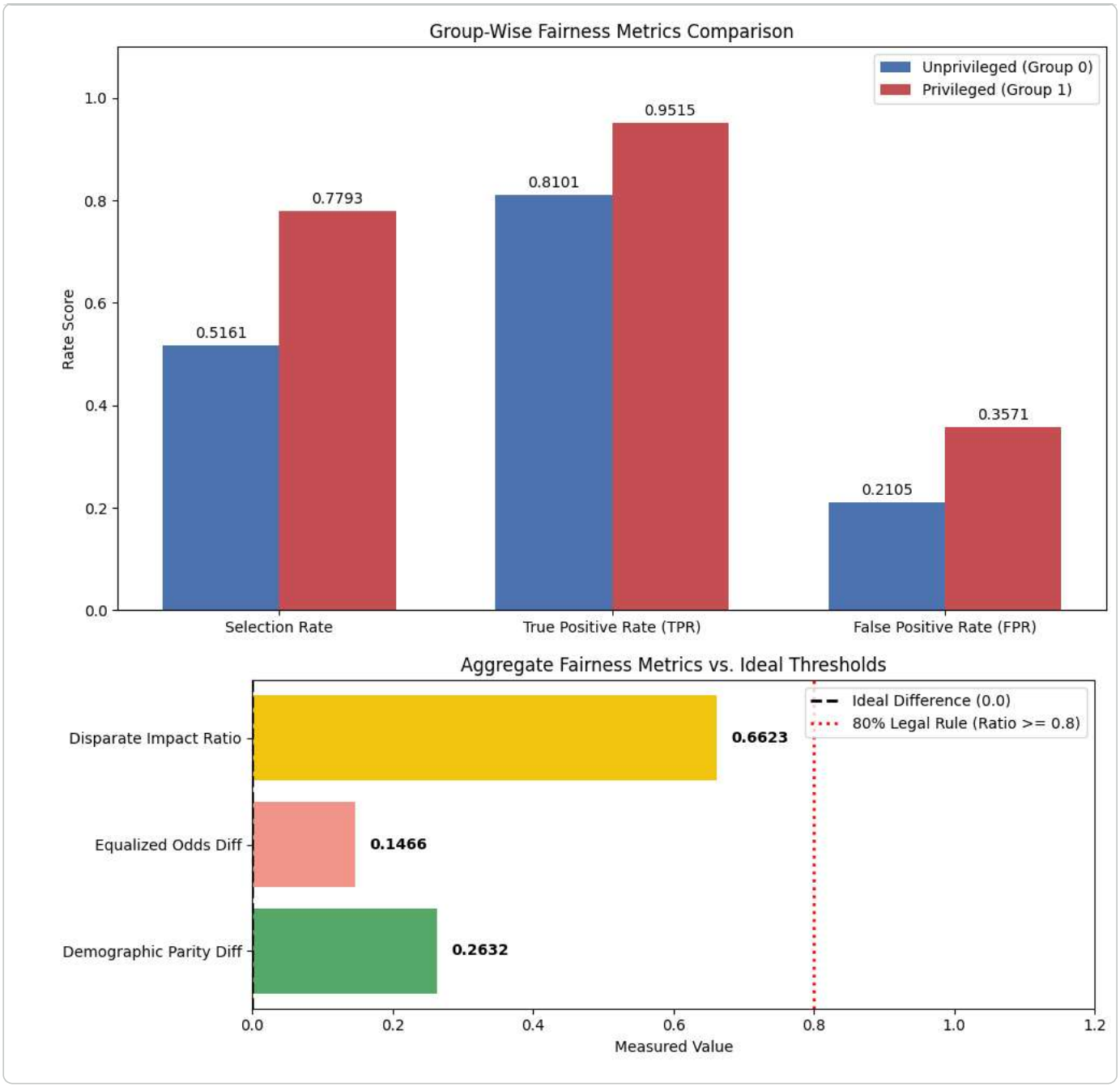
bars = ax.barh(metrics_names, metrics_values, color=['#55A868', '#F1948A', '#F1C40F'])

# Labeling and Formatting
ax.set_xlim(0, 1.2)
ax.set_xlabel('Measured Value')
ax.set_title('Aggregate Fairness Metrics vs. Ideal Thresholds')

# Add threshold lines to highlight bias
ax.axvline(x=0.0, color='black', linestyle='--', linewidth=2, label='Ideal Difference (0.0)')
ax.axvline(x=0.8, color='red', linestyle=':', linewidth=2, label='80% Legal Rule (Ratio >= 0.8)')
ax.legend()

# Annotate bars with exact values from your results
for i, v in enumerate(metrics_values):
    ax.text(v + 0.02, i, f"{v:.4f}", va='center', color='black', fontweight='bold')

plt.tight_layout()
plt.show()
```


Group-Wise Fairness Metrics Comparison
Unprivileged (Group 0)
Privileged (Group 1)
Rate Score
0.0
0.2
0.4
0.6
0.8
1.0
0.5161
0.7793
0.8101
0.9515
0.2105
0.3571
Selection Rate
True Positive Rate (TPR)
False Positive Rate (FPR)
Aggregate Fairness Metrics vs. Ideal Thresholds
Ideal Difference (0.0)
80% Legal Rule (Ratio >= 0.8)
Disparate Impact Ratio
Equalized Odds Diff
Demographic Parity Diff
0.6623
0.1466
0.2632
0.0
0.2
0.4
0.6
0.8
1.0
1.2
Measured Value